```
async function handleFile() {
    const fileInput = document.getElementById("fileInput");
    const file = fileInput.files[0];

    if (!file) {
      alert("Please upload a file.");
      return;
    }

    // Read and hash the file
    const arrayBuffer = await file.arrayBuffer();
    const hashBuffer = await crypto.subtle.digest("SHA-256", arrayBuffer);
    const hashArray = Array.from(new Uint8Array(hashBuffer));
    const hashHex = hashArray.map(b => b.toString(16).padStart(2, '0')).join('');

    document.getElementById("result").innerText = `File hash: ${hashHex}`;

    // Load Web3 and the contract
    if (typeof window.ethereum !== 'undefined') {
      const web3 = new Web3(window.ethereum);
      await window.ethereum.enable(); // Ask user to connect Metamask

      const contractAddress = "0xb9b1980900ED54a63eFd3A7CB98A7Ec1172D7E";
      const abi = [
        {
          "anonymous": false,
          "inputs": [
            {
              "indexed": false,
              "internalType": "bytes32",
              "name": "hash",
              "type": "bytes32"
            }
          ],
          "name": "EvidenceUploaded",
          "type": "event"
        },
        {
          "inputs": [
            {
              "internalType": "bytes32",
              "name": "hash",
              "type": "bytes32"
```

```
          }
        ],
        "name": "uploadReport",
        "outputs": [],
        "stateMutability": "nonpayable",
        "type": "function"
      },
      {
        "inputs": [
          {
            "internalType": "bytes32",
            "name": "hash",
            "type": "bytes32"
          }
        ],
        "name": "checkReport",
        "outputs": [
          {
            "internalType": "bool",
            "name": "",
            "type": "bool"
          }
        ],
        "stateMutability": "view",
        "type": "function",
        "constant": true
      }
    ];

    const contract = new web3.eth.Contract(abi, contractAddress);
    const accounts = await web3.eth.getAccounts();

    // Check if the hash exists on the blockchain
    const exists = await contract.methods.checkEvidence(hashHex).call({ from:
accounts[0] });

    if (exists) {
      document.getElementById("result").innerText += `\n Evidence is genuine
and matches the stored hash.`;
    } else {
      document.getElementById("result").innerText += `\n No match found. The
file may have been tampered with.`;
    }
```

```
    } else {
      alert("Please install MetaMask to use this app.");
    }
  }
```